

Questão 08 - Postmortem técnico de incidente em produção

Framework escolhido: R-I-S-E

Prompt (aplicando R-I-S-E)

[ROLE]

Você é um SRE Sênior atuando como Incident Commander técnico da Hill Valley Tech, responsável por produzir, durante um incidente em andamento, um postmortem técnico preliminar para Doc Brown (CTO), que está em call agora e precisa decidir em até 20 minutos entre duas ações concretas: (A) rollback do deploy chronos-api v2.48.0, ou (B) scaling emergencial (aumento de limits do RDS e do pool de conexões). Seu papel não é apenas descrever o incidente, é fundamentar uma recomendação objetiva com base nas evidências fornecidas.

[INPUT]

1) Changelog do deploy mais recente (ontem, 18:42 UTC):

Deploy chronos-api: v2.47.0 -> v2.48.0

Argo CD sync: 2026-04-23 18:42:11 UTC

Changelog:

- Adicionado endpoint POST /v2/transactions/batch
- Refatorado cliente do Ledger (pool de conexões movido para nova biblioteca interna)
- Bump de psycopg 3.1.18 -> 3.2.0
- Reduzido timeout do Ledger de 5s para 2s

2) Métricas do Beacon (últimos 30min):

timestamp | p99_latency_ms | req_rate_s | err_rate_pct

13:30 | 420 | 1200 | 0.2

13:45 | 510 | 1450 | 0.3

14:00 | 780 | 1780 | 0.8

14:10 | 2400 | 2100 | 4.5

14:15 | 5200 | 2400 | 8.2

14:20 | 8100 | 2650 | 11.7

3) Log do pod chronos-api-79c4d8b9-xk2jp:

14:19:48 [ERROR] [ledger-client] connection pool exhausted (max=20, active=20, waiting=147)

14:19:49 [WARN] [ledger-client] query timeout after 2000ms: SELECT ... FROM transactions WHERE ...

14:19:49 [ERROR] [handler] POST /v2/transactions/batch failed: context deadline exceeded

14:19:50 [ERROR] [ledger-client] connection reset by peer

14:19:51 [WARN] [circuit-breaker] ledger-client OPEN (threshold 50%, current 87%)

14:19:52 [ERROR] [reactor] failed to publish message: chronos-api upstream error

4) Estado do Reactor (fila chronos-transactions):

- 50.127 mensagens acumuladas, crescendo a ~800/min

- Consumer lag atual: 18 minutos e aumentando

5) Estado do cluster:

- Chronos: 12/12 pods running (HPA no máximo)

- CPU médio dos pods: 62%

- Memória média dos pods: 71%

- Conexões ativas ao Ledger: 240/250 (limite do RDS)

[STEPS]

Execute a análise nesta ordem, tornando cada etapa visível no documento final:

1. Construa uma linha do tempo correlacionando o horário do deploy (18:42

UTC de ontem) com o início da degradação nas métricas — identifique se a degradação começou logo após o deploy ou é mais recente/relacionada a pico de tráfego

2. Cruze o changelog do deploy com os erros do log: avalie especificamente

se a redução do timeout do Ledger (5s→2s) e a refatoração do pool de conexões são causa provável do "connection pool exhausted (max=20)" e do circuit breaker aberto

3. Descarte ou confirme explicitamente saturação de CPU/memória/HPA como causa raiz, usando os números do estado do cluster (62% CPU, 71% memória, 12/12 pods) — isto é decisivo para saber se scaling horizontal resolveria algo
4. Trate o acúmulo na fila do Reactor (50.127 mensagens, lag de 18min) como efeito ou causa — justifique com base na ordem dos eventos no log (o erro do reactor aparece antes ou depois dos erros do ledger-client?)
5. Avalie as duas ações candidatas (rollback vs. scaling emergencial) individualmente contra a causa raiz identificada: cada uma resolve o problema? Em quanto tempo? Que risco cada uma carrega?
6. Emita uma recomendação objetiva (não "depende") com nível de confiança, e liste as ações imediatas de mitigação, na ordem em que devem ser executadas

[EXPECTATION]

O documento precisa ser lido e compreendido por Doc Brown em menos de 3 minutos dentro da call — seja direto, sem rodeios narrativos. A recomendação final precisa ser inequívoca (uma ação clara, não as duas opções em pé de igualdade) e sustentada explicitamente pelos números fornecidos, nunca por suposição não verificável. Se a evidência for insuficiente para descartar totalmente uma hipótese, isso deve ser declarado como incerteza explícita, não omitido.

[FORMATO DE SAÍDA]

Markdown, seções: Timeline | Causa Raiz | Ligação Causal (evidência) | Rollback vs. Scaling (tabela comparativa) | Recomendação | Ações Imediatas.

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Output gerado

Postmortem Técnico Preliminar — Incidente Chronos API (14:20 UTC, 2026-04-24)

Timeline

- **2026-04-23 18:42 UTC** — Deploy v2.47.0 → v2.48.0 via Argo CD. Mudanças relevantes: refatoração do pool de conexões do Ledger (nova biblioteca interna) e redução do timeout de 5s para 2s.
- **13:30–14:00 UTC (hoje)** — Sistema estável, p99 em 420-780ms, erro <1%, crescimento orgânico de tráfego (pico esperado): 1200→1780 req/s.
- **14:10 UTC** — p99 salta para 2400ms, erro sobe para 4.5%. Início da degradação.
- **14:19:48–52 UTC** — Cascata de falhas: pool de conexões do Ledger esgotado (max=20, 147 requisições em espera) → timeouts de query (2000ms) → circuit breaker abre (87% de falha) → Reactor falha ao publicar mensagens.
- **14:20 UTC** — p99 em 8100ms, erro em 11.7%, fila do Reactor com 50.127 mensagens acumuladas e lag de 18min.

Observação-chave: a degradação não começou no momento do deploy (ontem), mas sim ~19h30min depois, coincidindo com o crescimento do tráfego de pico. Isso indica que o deploy **introduziu uma vulnerabilidade latente** que só se manifestou sob carga elevada — não um bug que quebra imediatamente.

Causa Raiz

Pool de conexões do Ledger subdimensionado combinado com timeout excessivamente agressivo, introduzidos no deploy v2.48.0.

Evidência:

- O log mostra `max=20` conexões no pool do cliente Ledger — um valor baixo para suportar 2650 req/s de pico, especialmente após a refatoração do

pool para a "nova biblioteca interna" citada no changelog

- O timeout foi reduzido de 5s para 2s no mesmo deploy — sob pico de tráfego, isso aumenta a taxa de queries que estouram o timeout antes de completar, prendendo conexões por mais ciclos e acelerando o esgotamento do pool
- **CPU (62%) e memória (71%) NÃO estão saturadas**, e os 12/12 pods do HPA já estão no teto — ou seja, o gargalo **não é capacidade computacional**, é especificamente o pool de conexões ao Ledger

Ligação Causal (evidência)

1. `connection pool exhausted (max=20, active=20, waiting=147)` — 14:19:48
2. `query timeout after 2000ms` — 14:19:49 (1s depois, consequência direta)
3. `POST /v2/transactions/batch failed: context deadline exceeded` — 14:19:49
4. `circuit-breaker OPEN` — 14:19:51 (proteção ativada após taxa de erro >50%)
5. `reactor failed to publish message: chronos-api upstream error` — 14:19:52
(**último da cadeia** — o Reactor falha **depois** do circuit breaker abrir, confirmando que o acúmulo na fila é **efeito**, não causa, do problema)

Conclusão: a fila do Reactor crescendo (50.127 mensagens, +800/min) é

consequência do Chronos estar rejeitando/atrasando processamento, não uma causa independente a ser tratada separadamente.

Rollback vs. Scaling Emergencial

| Critério | Rollback (v2.48.0 → v2.47.0) | Scaling emergencial (RDS + pool) |

| --- | --- | --- |

| Ataca a causa raiz? | **Sim** — remove o timeout agressivo (2s) e a refatoração do pool que introduziu o problema | **Não diretamente** — aumenta capacidade, mas não corrige o timeout de 2s nem valida se a nova biblioteca de pool tem bug |

| Tempo estimado até mitigação | 5-10 min (rollback via Argo CD já testado em produção antes) | Incerto — aumentar limite do RDS pode exigir reboot/parameter group change; conexões já em 240/250, próximo do teto físico |

| Risco | Baixo — v2.47.0 era estável e validada em produção | Médio-alto — sobe o teto sem saber se o vazamento/uso ineficiente de conexões persiste; pode apenas adiar o esgotamento |

| Efeito colateral | Perde-se temporariamente o endpoint `/v2/transactions/batch` (novo nesta versão) | Nenhuma perda de funcionalidade, mas não resolve a causa: erro pode recorrer no próximo pico |

| Reversibilidade | Alta (rollback é reversível via novo deploy) | Baixa no curto prazo (mudança de infraestrutura RDS não é instantânea nem trivial de reverter sob incidente) |

Recomendação

****Rollback imediato para v2.47.0.**** Confiança: ****alta****. A causa raiz está diretamente ligada a duas mudanças específicas do deploy v2.48.0 (timeout 2s + novo pool), CPU/memória não são o gargalo (descartando a tese de que scaling resolveria), e o rollback é a ação de menor risco e maior reversibilidade disponível dentro da janela de 20 minutos.

Ações Imediatas

1. Executar rollback via `argocd app rollback chronos-api <revisão v2.47.0>`
2. Monitorar `connection pool` e `p99\_latency\_ms` nos 5 minutos seguintes para confirmar recuperação
3. Não escalar RDS neste momento — evidência não sustenta que seja a causa
4. Após estabilização, o Reactor deve drenar a fila naturalmente; monitorar consumer lag para confirmar queda gradual
5. Abrir investigação assíncrona (pós-incidente) sobre a nova biblioteca de pool de conexões antes de reintroduzir v2.48.0

Justificativa estendida — R-I-S-E e comparação com outros frameworks

Como R-I-S-E aparece no prompt

Componente	Onde aparece	Por quê
Role	"SRE Sênior atuando como Incident Commander... seu papel não é apenas descrever o incidente, é fundamentar uma recomendação"	Ancora a IA numa postura de decisão , não de relatório passivo — essencial porque Doc Brown precisa de uma posição, não de uma lista neutra de fatos
Input	Os 5 artefatos completos (changelog, métricas, log, fila, cluster) organizados e rotulados	Sem estruturar o Input explicitamente, a IA tende a tratar as fontes de forma solta; aqui elas viram blocos de evidência endereçáveis, o que facilita a etapa seguinte de correlação
Steps	6 passos ordenados: timeline → cruzamento causal → descarte de hipótese (CPU/memória) → causa vs. efeito da fila → avaliação das duas ações → recomendação	É o componente mais crítico deste prompt — obriga a IA a descartar explicitamente a hipótese de saturação computacional (passo 3) antes de recomendar, evitando o erro mais comum em diagnóstico sob pressão: escalar recursos sem confirmar que é isso que está faltando
Expectation	"lido em menos de 3 minutos... recomendação inequívoca... sustentada pelos números... incerteza declarada, não omitida"	Traduz a restrição real do cenário (20 minutos, decisão ao vivo) em critério de qualidade do texto — impede que a IA entregue um "depende" elegante, que seria inútil numa call de incidente

Comparação com T-A-G

T-A-G também seria defensável — poderíamos mapear Task = "produza o postmortem", Action = os mesmos passos analíticos, Goal = "permitir que Doc decida em 20 minutos". Na prática, o **Action** do T-A-G cobriria quase o mesmo terreno que o **Steps** do R-I-S-E.

O que se **perderia** com T-A-G:

- T-A-G não tem um componente **Role** nativo. Isso importa aqui porque o documento vai ser lido em voz alta/rapidamente numa call de decisão — a persona de "Incident Commander que precisa recomendar, não só descrever" muda o tom do output. Sem Role, a tendência natural do modelo é produzir um relatório mais descritivo e menos assertivo (no Q3, onde usei T-A-G, isso não era um problema porque o objetivo ali era só apresentar oportunidades priorizadas, não tomar uma decisão binária sob pressão)
- O **Goal** do T-A-G tende a expressar uma *finalidade única* ("reduzir 15% de custo", "fundamentar a meta"). Aqui, porém, existem **duas ações concretas e mutuamente exclusivas** para comparar (rollback vs. scaling) — isso pede uma etapa de avaliação comparativa explícita, que o **Expectation** do R-I-S-E cobre melhor ao exigir

"recomendação inequívoca, não as duas opções em pé de igualdade". No T-A-G, essa exigência ficaria diluída dentro do Goal, com risco de a IA apresentar as duas opções lado a lado sem se comprometer com uma

O que se **ganharía** com T-A-G: é um framework mais enxuto — para um cenário com uma causa raiz óbvia e uma única ação recomendável, T-A-G entregaria o essencial com menos texto de prompt. Mas neste caso, a ambiguidade entre duas ações concretas é o cerne do problema, e é justamente isso que pede a separação mais rígida entre Steps (processo) e Expectation (padrão de decisão) que o T-A-G não oferece.

Comparação com C-A-R-E

C-A-R-E foi meu segundo candidato descartado, e o motivo é estrutural: o componente **Example** não tem para onde apontar aqui. Nas Questões 6 (módulo Terraform), havia um artefato de referência real (o módulo de VPC) para a IA espelhar estilo. Neste incidente, **não existe um postmortem anterior fornecido como padrão de referência** — forçar um Example inventado (ex.: "escreva como um postmortem típico do Google SRE Book") adicionaria uma referência genérica e desnecessária, sem ganho real, e desviaria o foco do que importa: a evidência específica deste incidente.

O que se **perderia** com C-A-R-E: o **Context** dele absorveria bem os 5 artefatos (papel parecido com o Input do R-I-S-E), e o **Result** capturaria bem o critério de sucesso ("aprovável por Doc Brown, decisão fundamentada"). Mas sem Steps explícitos — C-A-R-E tem só **Action**, um bloco único — corre-se o risco de a IA pular direto para a recomendação sem demonstrar o raciocínio de descarte de hipóteses (CPU/memória) que é justamente o que dá credibilidade técnica ao postmortem numa situação onde Doc pode questionar "por que não é só falta de recurso?".

O que se **ganharía** com C-A-R-E: nada de relevante neste caso específico, já que seu diferencial (ancoragem em exemplo de estilo) não tem aplicação no cenário — é um indício de que C-A-R-E é o framework certo quando existe consistência de padrão a preservar (como no módulo Terraform), não quando o desafio é diagnóstico analítico original.

Por que R-T-F e B-A-B foram descartados rapidamente

- **R-T-F** é bom demais para artefatos autocontidos e simples (Dockerfile, script) — não tem componente para lidar com **múltiplas fontes de evidência correlacionadas** nem para impor uma sequência de descarte de hipóteses. Usá-lo aqui produziria um prompt raso demais para a complexidade analítica exigida.
- **B-A-B** pressupõe uma transição de estado com destino único e já conhecido (como modernizar o Deployment do Chronos, Questão 5). Aqui, o "After" não está definido de antemão — é justamente o que a análise precisa **descobrir** (rollback ou scaling?). Forçar B-A-B tenderia a enviesar a IA para prescrever uma correção específica antes mesmo de comparar as duas alternativas, o que compromete a neutralidade analítica que Doc Brown precisa para decidir.